
15.1. Presentación del capítulo

En este apartado vemos algunas de las características más avanzadas de los diagramas de actividad. Éstas son características que no es probable que utilice todos los días pero que pueden ser de mucha utilidad en ciertas situaciones de modelado.

Los apartados en este capítulo se pueden leer en cualquier orden. También puede saltarse este capítulo y luego consultar el apartado apropiado cuando necesite utilizar una característica específica.

15.2. Conectores

Como principio general debería evitar utilizar conectores. Sin embargo, si encontrara una complejidad irreducible, puede utilizar conectores para romper extremos largos que son difíciles de seguir. Esto puede simplificar un diagrama de actividad y hacerlo más sencillo de leer.

La sintaxis del conector se muestra en la figura 15.2. Para una actividad dada, cada conector de salida debe tener exactamente un conector entrante con la misma etiqueta. Las etiquetas son identificadores para el conector y no tienen otra semántica. A menudo simplemente son letras del alfabeto.

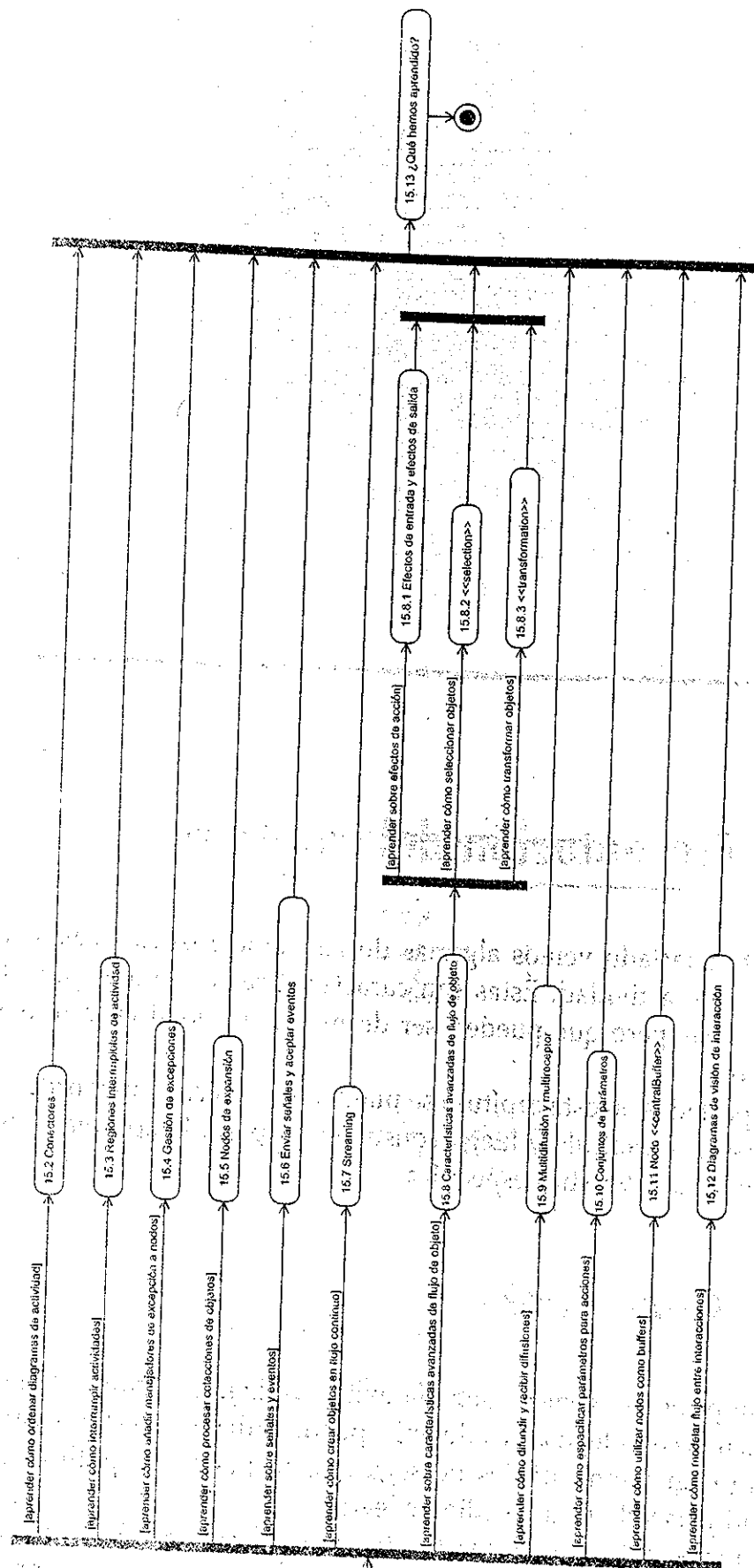


Figura 15.1.

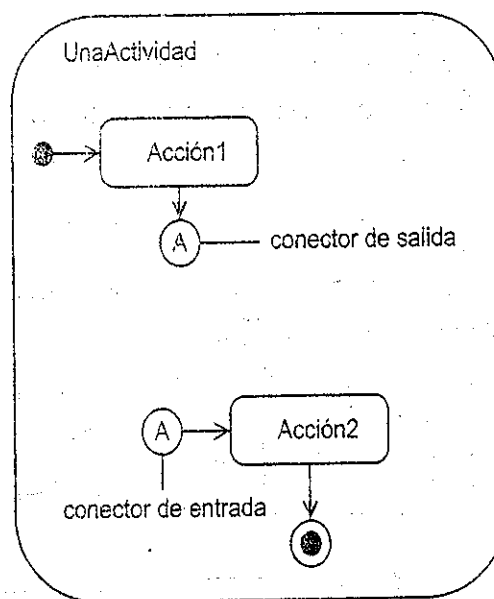


Figura 15.2.

15.3. Regiones interrumpibles de actividad

Las regiones interrumpibles de actividad son regiones de una actividad que se interrumpen cuando un token atraviesa un extremo que se interrumpe. Cuando la región se interrumpe, todo el flujo dentro de la región se aborta inmediatamente. Las regiones interrumpibles de actividad le proporcionan un medio de utilidad de modelar interrupciones y eventos asíncronos. Se utilizan más a menudo en diseño, pero también se pueden utilizar para mostrar la gestión de eventos asíncronos de negocio. La figura 15.3 muestra una sencilla actividad Conectarse que tiene una región interrumpible. La región se muestra como un rectángulo redondeado de los puntos que engloba las acciones Obtener NombreUsuario, Obtener Contraseña y Cancelar. Si la acción de aceptar evento Cancelar obtiene el evento Cancelar mientras el control está en la región, muestra un token en un extremo de interrupción e interrumpe la región. Las acciones Obtener NombreUsuario, Obtener Contraseña y Cancelar, todas se terminan. Los extremos de interrupción se dibujan como una flecha de zigzag según se muestra en la figura 15.3 o como una flecha normal con un icono de zigzag dibujado sobre ésta según se muestra en la figura 15.4.

15.4. Gestión de excepciones

Los lenguajes informáticos modernos a menudo gestionan los errores por medio de un mecanismo denominado gestión de excepciones. Si se detecta un error en una parte protegida de código, se crea un objeto de excepción y el flujo de control salta hasta un manejador de excepción que procesa el objeto de excepción de alguna

forma. El objeto de excepción contiene información sobre el error que se puede utilizar por el manejador de excepción. El manejador de excepción puede terminar la aplicación o tratar de recuperarse. La información en el objeto de excepción a menudo se guarda en un registro de error.

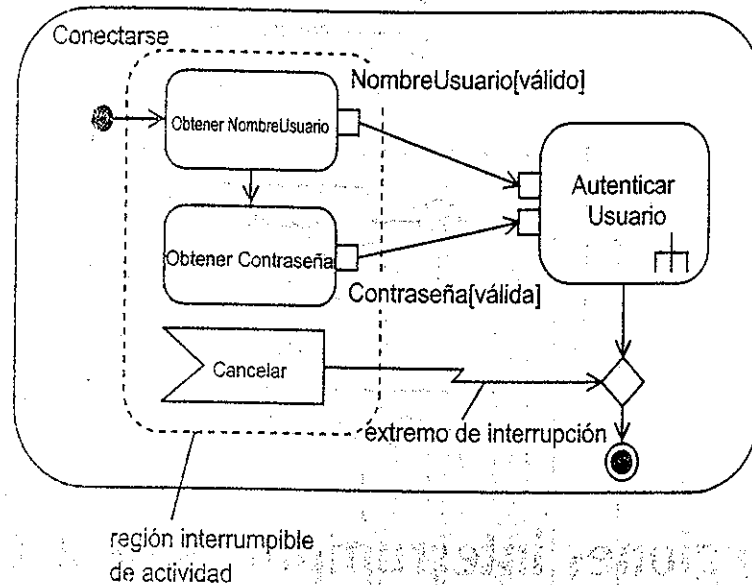


Figura 15.3.

Figura 15.4.

Puede modelar esta gestión de excepción en diagramas de actividad al utilizar pins de excepción, nodos protegidos y manejadores de excepción. En la figura 15.5 hemos actualizado nuestra actividad Conectarse para hacer que la actividad Autenticar usuario tenga como resultado un objeto RegistrarExcepcion si el usuario no se puede autenticar. Este objeto se consume por la acción Registrar error que escribe la información de error en un registro de error. Puede mostrar que un pin representa el resultado de un objeto de excepción al anotarlo con un pequeño triángulo equilátero según se muestra en la figura. El nodo Registrar error actúa como un manejador de excepción que procesa las excepciones generadas por Autenticar usuario. Cuando un nodo tiene un manejador de excepción asociado, se conoce como un nodo protegido. Puesto que la gestión de excepción es normalmente una cuestión de diseño en lugar de análisis, tiende a utilizar nodos protegidos más en diseño que en análisis. Sin embargo, algunas veces puede ser de utilidad modelar un nodo protegido en análisis si tiene semántica importante de negocio.

15.5. Nodos de expansión

Los nodos de expansión le permiten mostrar cómo una colección de objetos se procesa por una parte del diagrama de actividad denominada región de expansión.

Ésta puede ser una técnica de utilidad tanto en análisis como en diseño ya que de lo contrario puede ser bastante difícil mostrar cómo se procesa una colección.

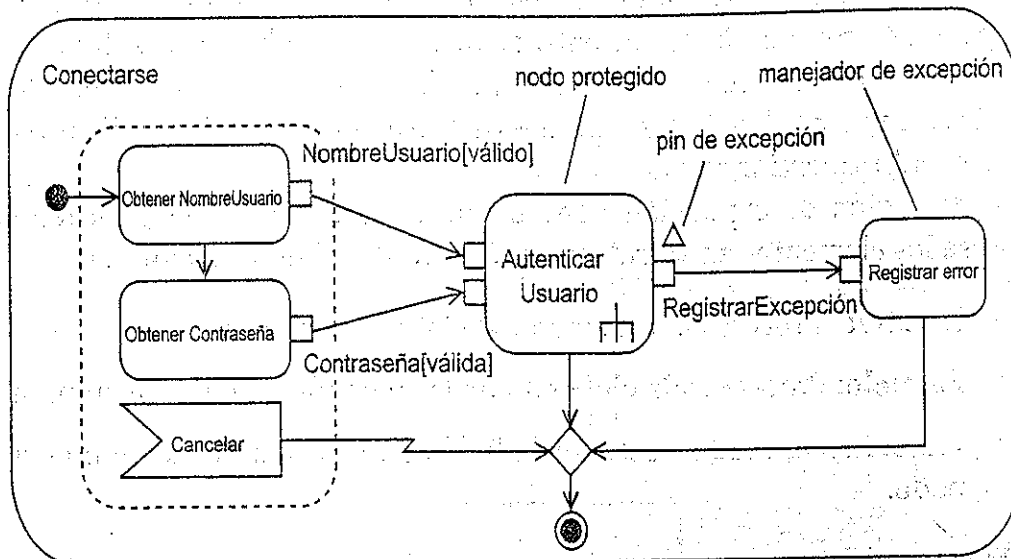


Figura 15.5.

Un nodo de expansión es un nodo de objeto que representa una colección de objetos que fluyen en o fuera de una región de expansión. La región de expansión se ejecuta una vez por elemento de entrada.

La figura 15.6 muestra un ejemplo de una región de expansión, mostrada como un rectángulo redondeado de puntos con nodos de expansión de entrada y salida. Un nodo de expansión se parece a un pin pero con tres cuadros. Esta representación es para indicar que acepta una colección en lugar de un solo objeto.

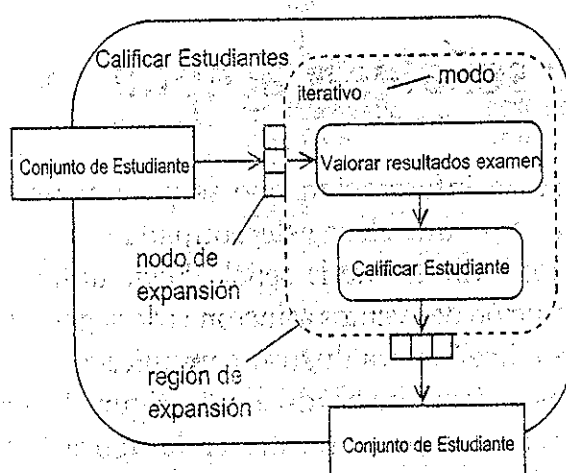


Figura 15.6.

Existen dos restricciones en los nodos de expansión:

- El tipo de colección de salida debe coincidir con el tipo de la colección de entrada.

- El tipo del objeto contenido en las colecciones de entrada y salida deben ser iguales.

Estas restricciones significan que las regiones de expansión no se pueden utilizar para transformar objetos de entrada de un tipo en objetos de salida de otro tipo.

El número de colecciones de salida puede ser diferente del número de colecciones de entrada, por lo que las regiones de expansión se pueden utilizar para combinar o dividir colecciones.

Toda región de expansión tiene un modo que determina el orden en el que procesa los elementos en su colección de entrada. El nodo puede ser:

- **Iterativo:** Procesa cada elemento de la colección de entrada secuencialmente.
- **Paralelo:** Procesa cada elemento de la colección de entrada en paralelo.
- **Flujo:** Procesa cada elemento de la colección de entrada a medida que llega al nodo.

Siempre debe especificar explícitamente el modo ya que la especificación UML no define ningún modo por defecto.

En la figura 15.6 la región de expresión acepta un conjunto de objetos Estudiante. Procesa cada uno de estos objetos por turnos (modo = iterativo) y muestra como resultado un conjunto de objetos Estudiante procesados. Las dos acciones dentro de la región primero valoran los resultados del examen de un Estudiante y luego asignan una nota. En este caso, Conjunto de Estudiante solamente se ofrece en el nodo de expansión de salida cuando todos los Estudiantes se han procesado. Sin embargo, si el nodo fuera de flujo, entonces, cada objeto Estudiante se ofrecería en el nodo de expansión de salida tan pronto como se procesara.

15.6. Enviar señales y aceptar eventos

Una señal representa información que se pasa asincrónicamente entre objetos. Una señal se modela como una clase estereotipada <<signal>>. La información a pasar se alberga en los atributos de la señal. Puede utilizar señales en análisis para mostrar el envío y recepción de eventos asíncronos de negocio (como PedidoRecibido) y puede utilizarlos en diseño para ilustrar comunicación asíncrona entre diferentes sistemas, subsistemas o piezas de hardware. La figura 15.7 muestra dos señales que se utilizan en la actividad que se muestra en la figura 15.8. Estas dos señales son tipos de *EventoSeguridad*. La señal *EventoPeticiónAutorización* alberga un PIN y detalles de tarjeta. Éstos están probablemente cifrados. El *EventoAutorización* alberga un indicador booleano para indicar si la tarjeta y el PIN se han autorizado.

Puede enviar una señal al utilizar un nodo de acción de enviar señal. Esto envía la señal asincrónicamente; la actividad de envío no espera a la confirmación del recibo de la señal. La semántica de la acción de enviar señal es la siguiente:

- La acción de enviar señal se inicia cuando existe un token simultáneamente en todos los extremos de entrada. Si la señal tiene pins de entrada, debe recibir un objeto de entrada del tipo correcto para cada uno de sus atributos.
- Cuando la acción se ejecuta, se construye y envía un objeto de señal. El objeto destino normalmente no se especifica, pero si necesita especificarlo, puede pasarlo en la acción de enviar señal en un pin de entrada.
- La acción de envío no espera por la confirmación del recibo de señal; es asíncrono.
- La acción termina y los token de control se ofrecen en sus extremos de salida.

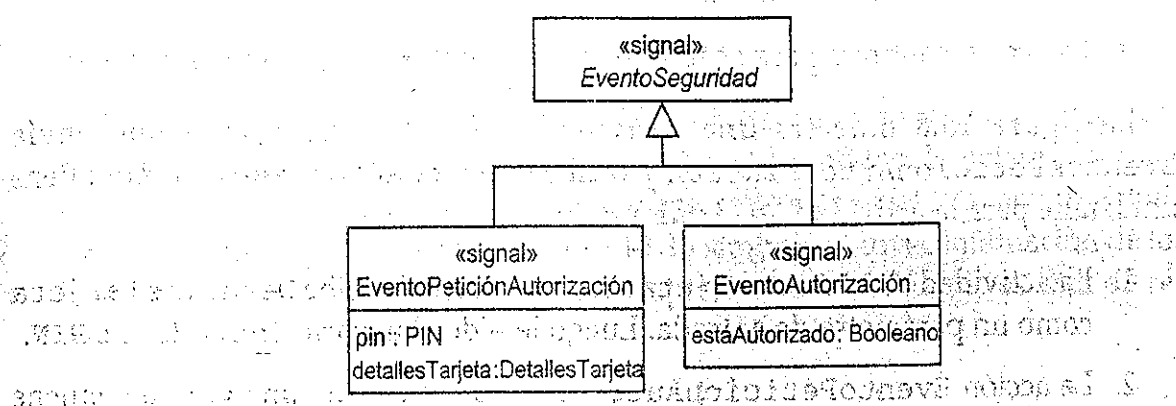


Figura 15.7.

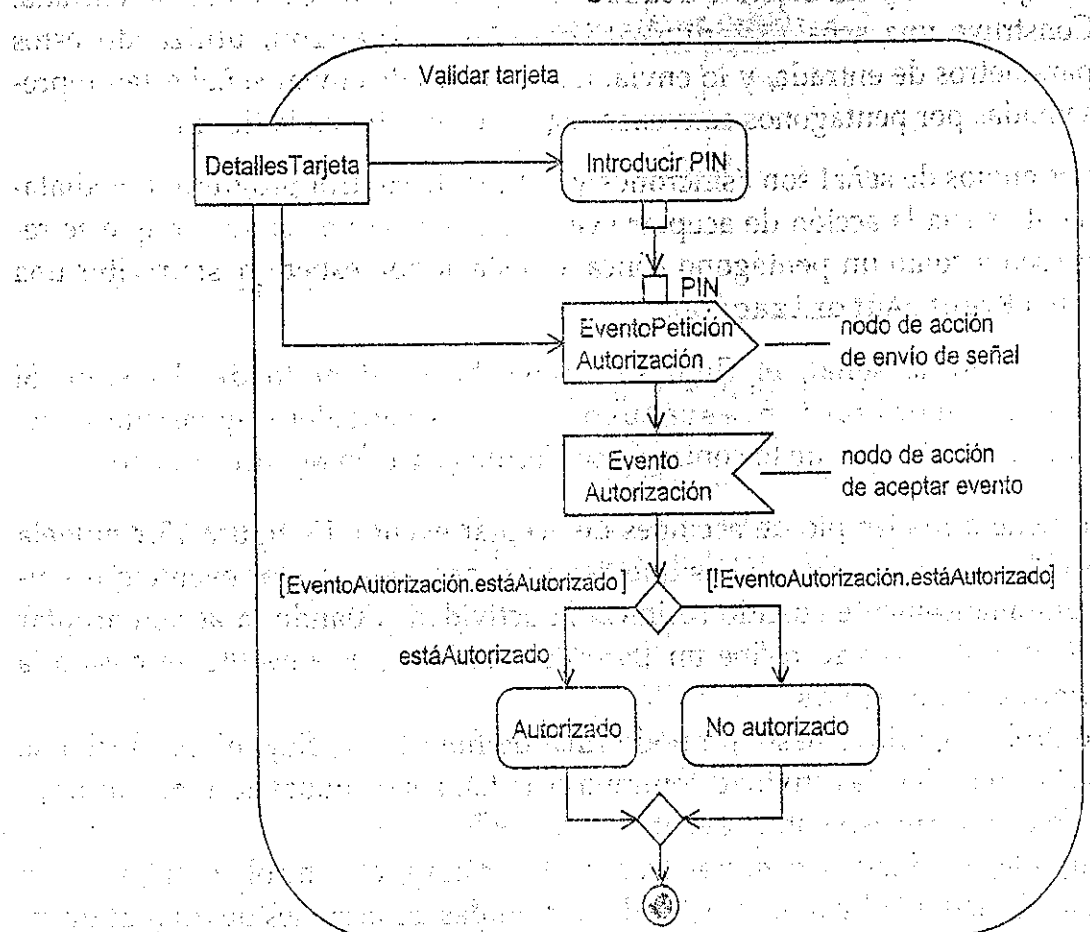


Figura 15.3.

Un nodo de acción de aceptar evento tiene cero o un extremo de entrada. Espera eventos asíncronos detectados por el contexto y los ofrece en su único extremo de salida. Tiene la siguiente semántica:

- La acción de aceptar evento se inicia por un extremo de control entrante, y si no tiene extremo entrante, se inicia cuando se inicia la actividad que lo posee.
- La acción espera a recibir un evento del tipo especificado. Este evento se conoce como el activador.
- Cuando la acción recibe un activador de evento del tipo correcto, muestra como salida un token que describe el evento. Si el evento era un evento de señal, el token es una señal.
- La acción continúa para aceptar eventos mientras se ejecuta la actividad.

La figura 15.8 muestra una actividad Validar tarjeta que envía EventosPeticiónAutorización y recibe EventosAutorización. Aquí tiene un detalle para la actividad Validar tarjeta.

1. La actividad Validar tarjeta empieza cuando recibe DetallesTarjeta como un parámetro de entrada. Luego le pide al usuario Introducir PIN.
2. La acción EventoPeticiónAutorización se ejecuta una vez que obtiene el objeto PIN y un objeto DetallesTarjeta en sus extremos de entrada. Construye una señal EventoPeticiónAutorización, utilizando estos parámetros de entrada, y lo envía. Las acciones de enviar señal están representadas por pentágonos convexos según se muestra en la figura.
3. Los envíos de señal son asíncronos y el flujo de control progresa inmediatamente hacia la acción de aceptar evento EventoAutorización que se representa como un pentágono cóncavo. Esta acción espera hasta recibir una señal EventoAutorización.
4. Al recibir la señal, el flujo se mueve hasta el nodo de decisión. Si EventoAutorización.estaAutorizado es verdadero, se ejecuta la acción Autorizado, de lo contrario se ejecuta la acción No autorizado.

Aquí tiene otro ejemplo de acciones de aceptar evento. La figura 15.9 modela una actividad Mostrar noticias que tiene dos acciones aceptar evento que empiezan automáticamente cuando se inicia la actividad. Cuando la acción aceptar evento EventoNoticias recibe un EventoNoticias, este evento se pasa a la acción Mostrar noticias.

El control luego fluye hasta un nodo final de flujo y este flujo en particular se termina. Sin embargo, la actividad continúa ejecutándose y ambas acciones de aceptar evento continúan esperando eventos.

Cuando la actividad obtiene un EventoTerminar, el control se mueve a un nodo final de actividad y toda la actividad, incluidas las acciones de aceptar evento, terminan inmediatamente.

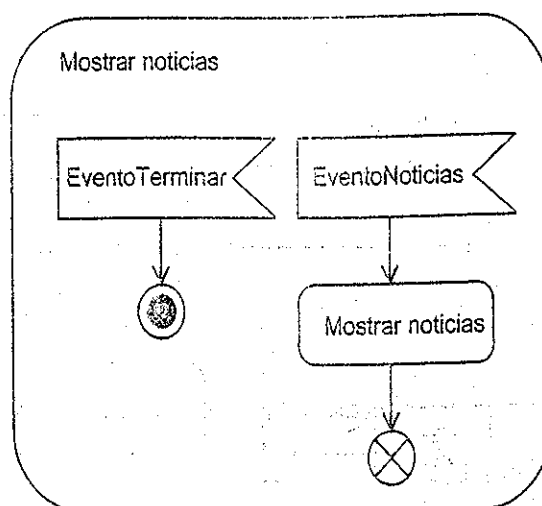


Figura 15.9.

En caso de que necesitara mostrar cómo las señales se mueven alrededor de diagramas de actividad, puede representarlas como nodos de objeto. Aunque las señales tienen la misma semántica que otros nodos de objeto, tienen una sintaxis especial. Esto se muestra en la figura 15.10. El símbolo es una combinación de los de la acción de enviar señal y la acción de aceptar evento, por lo que es fácil de recordar.

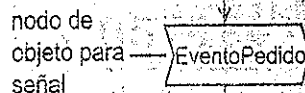


Figura 15.10.

15.7. Streaming

Las acciones normalmente solamente quitan tokens de sus extremos de entrada cuando empiezan a ejecutarse y los ofrecen a sus extremos de salida cuando han terminado. Sin embargo, algunas veces necesita una acción que se ejecute continuamente mientras acepta y ofrece tokens periódicamente. Este comportamiento se denomina streaming, y en UML 2 puede ilustrarlo de cualquiera de las cuatro formas que se muestran en la figura 15.11.

De hecho pensamos que son demasiadas formas para representarlo y le recomendamos que decida una forma y se ajuste a ella. La opción 1 es la opción más concisa y es la que recomendamos (pins en negro).

El ejemplo en la figura 15.11 ilustra un uso típico de streaming. La acción Leer puerto ratón lee continuamente el puerto del ratón y ofrece información sobre la actividad del ratón como EventosRatón pasada en flujo continuo en su extremo de salida. Estos EventosRatón se consumen por la acción Gestionar EventoRatón.

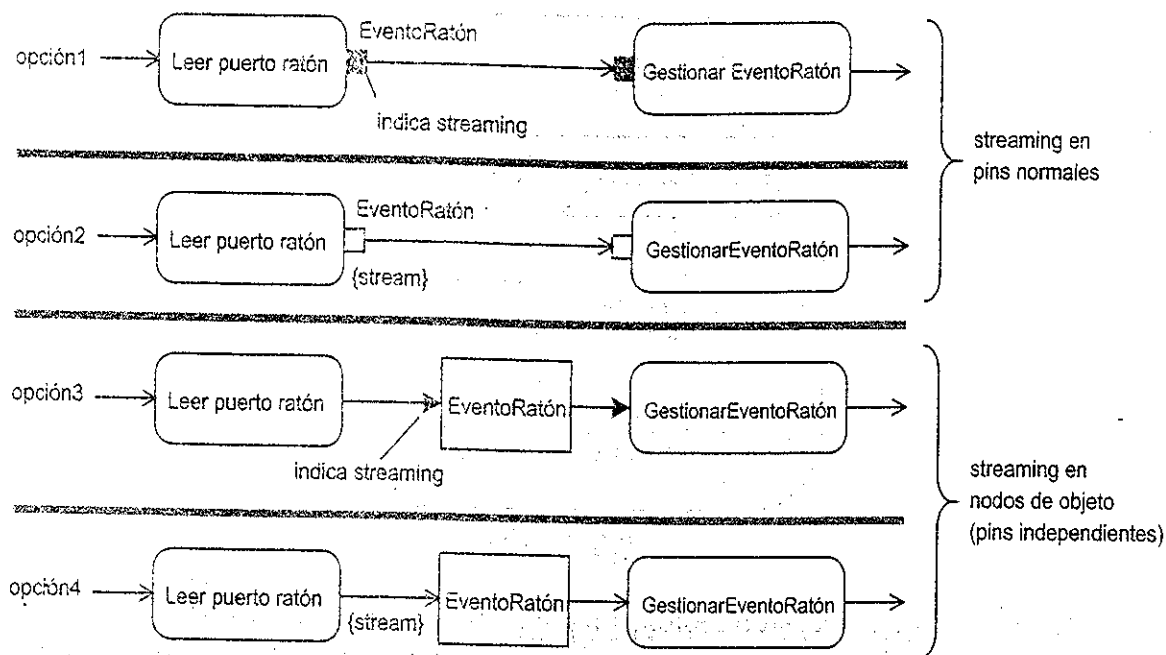


Figura 15.11.

Cualquier situación en la que necesite recibir y procesar información continuamente es un buen candidato para el streaming.

15.8. Características avanzadas de flujo de objeto

En este apartado examinamos algunas características avanzadas de flujo de objeto. Probablemente no necesitara utilizar estas características demasiado a menudo pero siempre es bueno saber que existen en caso de que las necesite. Hemos resumido estas características en la figura 15.12.

15.8.1. Efectos de entrada y efectos de salida

Los efectos de entrada y los efectos de salida muestran los efectos que tiene una acción sobre los objetos que entran o salen. Muestre estos efectos al situar una breve descripción del efecto entre llaves tan cerca del pin de entrada o salida como pueda. Existe un ejemplo de un efecto de entrada cerca de la acción Enviar Recibo en la figura 15.12. Este efecto especifica que la acción Enviar Recibo sella cada recibo que recibe. La acción Aceptar Pago tiene un efecto de salida {crear}. Esto indica que Aceptar Pago crea todo objeto Transacción que muestra como salida.

15.8.2. <<selection>>

Una selección es una condición anexada a un flujo de objeto que le lleva a aceptar solamente aquellos objetos que satisfacen la condición. La condición de selección se muestra en una nota estereotipada <<selection>>.

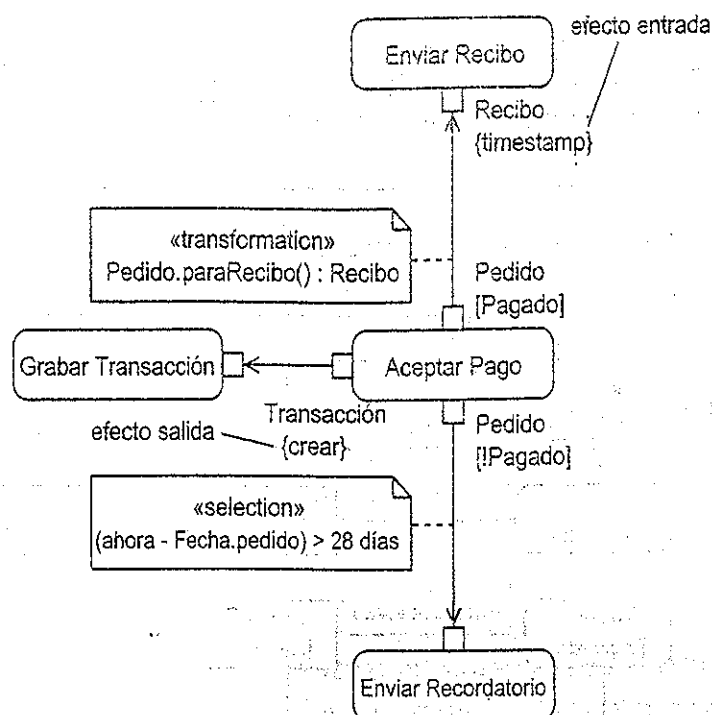


Figura 15.12.

En la imagen 15.12, puede ver la selección anexada al flujo de objeto entre Aceptar Pago y Enviar Recordatorio. Esta condición de selección es (ahora - Fecha.pedido) > 28 días. El pin de salida en Aceptar Pago ofrece objetos Pedido que están en el estado ! Pagado y la selección acepta solamente aquellos objetos Pedido que han estado pendientes durante más de 28 días. El efecto de este flujo es seleccionar todos los Pedidos impagados que han estado pendientes durante más de 28 días y pasarlos a la acción Enviar Recordatorio.

15.8.3. <<transformation>>

Una transformación transforma objetos en un flujo de objetos en objetos de un tipo diferente. La expresión de transformación se muestra en una nota estereotipada <<transformation>>. En la figura 15.12 puede ver una transformación anexada al flujo de objeto entre Aceptar Pago y Enviar Recordatorio. Esta especifica que los objetos Pedido mostrados como salida por Aceptar Pago se transformarán en los objetos Recibo esperados por Enviar Recibo. En este ejemplo, la transformación se realiza al invocar la operación paraRecibo() en cada objeto Pedido. Esta operación toma la información en el Pedido y crea un objeto Recibo. Utilice transformaciones cuando necesite conectar un pin de salida para instancias de un clasificador con un pin de entrada que espera instancias de un clasificador diferente.

15.9. Multidifusión y multireceptor

Normalmente, un objeto se envía a un receptor. Sin embargo, algunas veces necesita mostrar que un objeto se envía a múltiples receptores. Esto se denomina

multidifusión. De forma similar, podría necesitar mostrar que los objetos se reciben de múltiples emisores. Esto se denomina multirreceptor. Multidifusión y multirreceptor ocurren a menudo en pares simétricos como se muestra en la figura 15.13.

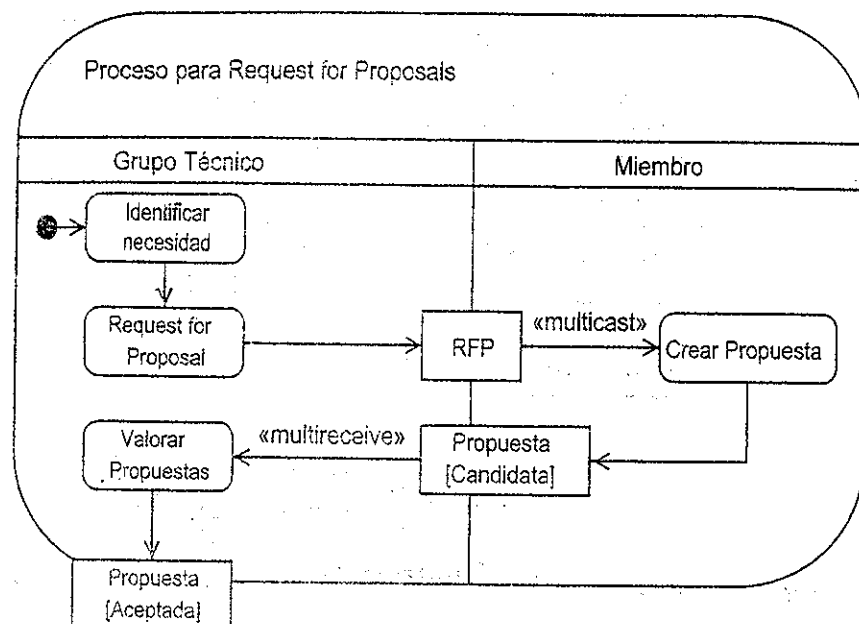


Figura 15.13.

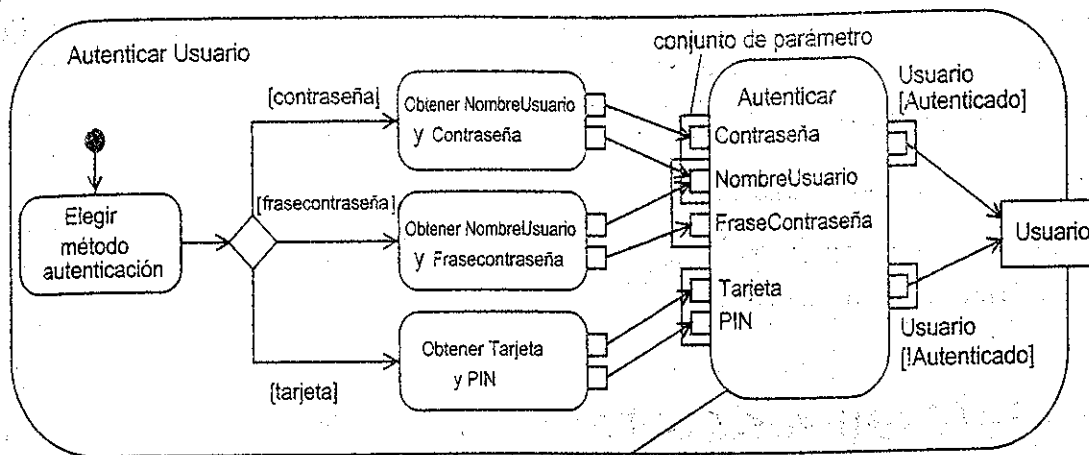
Para indicar que el flujo de un objeto se origina de un multidifusor o parte de un multirreceptor, estereotipa el flujo según se ilustra en la figura 15.13. Esta figura muestra un sencillo proceso de negocio similar al que la OMG utiliza para solicitar propuestas para sus estándares. El Grupo Técnico identifica primero la necesidad de una solución y luego formula ésta en una RFP (*Request For Proposal*). Esto es una multidifusión a muchos Miembros que crean propuestas. Estas son multirrecibidas por el grupo técnico que las evalúa y por último muestra una Propuesta Aceptada.

15.10. Conjuntos de parámetros

Los conjuntos de parámetros permiten que una acción tenga conjuntos alternativos de pins de entrada y salida. Estos conjuntos de pins se denominan conjuntos de parámetro. Los conjuntos de parámetro de entrada contienen pins de entrada y los conjuntos de parámetro de salida contienen pins de salida. No puede tener un conjunto mezclado de pins de entrada y salida.

Para ilustrar por qué querría utilizar conjuntos de parámetro, considere la figura 15.14. En esta figura existen tres tipos de acción de autenticación.

1. Obtener NombreUsuario y Contraseña, muestra como salida un objeto NombreUsuario y una Contraseña.
2. Obtener NombreUsuario y Frasecontraseña, muestra como resultado un objeto NombreUsuario y Frasecontraseña.
3. Obtener Tarjeta y PIN, muestra como resultado un objeto tarjeta y PIN.



condición de entrada: (NombreUsuario AND Contraseña) XOR (NombreUsuario AND FraseContraseña) XOR (Tarjeta AND PIN)
 salida: (Usuario [Autenticado]) XOR (Usuario [!Autenticado])

Figura 15.14.

Como puede ver, cada una de estas acciones muestra como resultado un conjunto diferente de objetos. Una solución a este problema sería tener una acción aparte de autenticar para cada conjunto de objetos tal como AutenticarNombreUsuarioYContraseña, AutenticarNombreUsuarioYFraseContraseña, y AutenticarTarjetaYPIN. Ésta es una solución razonable, pero presenta un par de desventajas:

- Hace que el diagrama de actividad sea bastante verboso.
- La autenticación está ahora distribuida en tres acciones en lugar de estar localizada en un sitio.

Utilizando conjuntos de parámetros, puede crear una sola acción Autenticar que puede gestionar los tres conjuntos diferentes de parámetros de entrada.

En la figura 15.14 puede ver que Autenticar tiene tres conjuntos de parámetros de entrada y dos de salida. Los conjuntos de parámetros de entrada son:

- NombreUsuario AND Contraseña.
- NombreUsuario AND FraseContraseña.
- Tarjeta AND PIN.

Solamente uno de estos conjuntos de parámetro de entrada se puede utilizar por ejecución de la acción. Existe por lo tanto, una relación XOR entre ellos. Observe que los conjuntos {NombreUsuario, Contraseña} y {NombreUsuario, FraseContraseña} tienen el pin NombreUsuario en común. Como muestra la figura, puede indicar esto al solapar los dos conjuntos de parámetro de modo que los pin comunes estén en la región solapada. Sin embargo, esto puede hacerse bastante confuso por lo que podría elegir tener dos pin NombreUsuario, uno en cada conjunto de parámetros de entrada. Nosotros preferimos la última solución y el diagrama, según está, es para ilustrar la sintaxis de los pin solapados en caso de que deseara utilizarlos.

La acción Autenticar tiene los siguientes conjuntos de parámetros de salida, cada uno de los cuales contiene un solo pin:

- Usuario[Autenticado]
- Usuario[!Autenticado]

Nuevamente, solamente uno de estos conjuntos de parámetros de salida se utilizará por ejecución de la actividad.

15.11. Nodo <<centralBuffer>>

Como se ha mencionado en el capítulo 14, todos los nodos de objeto tienen posibilidades de buffer.

Los nodos centrales de buffer son nodos de objeto que se utilizan específicamente como buffers entre flujos de objeto de entrada y salida. Le permiten combinar múltiples flujos de objeto de entrada y distribuir los objetos entre múltiples flujos de objeto de salida. Cuando se utiliza un nodo de objeto como un buffer central, se le debería asignar el estereotipo <<centralBuffer>>.

La figura 15.15 muestra un sencillo ejemplo en el que se utiliza un buffer central para acumular los diferentes tipos de objeto Pedido en flujo continuo desde múltiples canales de venta. Los objetos Pedido se mantienen en el buffer mientras esperan que la acción Procesar pedido los acepte. El buffer central Pedido puede albergar objetos del tipo Pedido o sus subclases PedidoWeb, PedidoTelefono y PedidoCorreo.

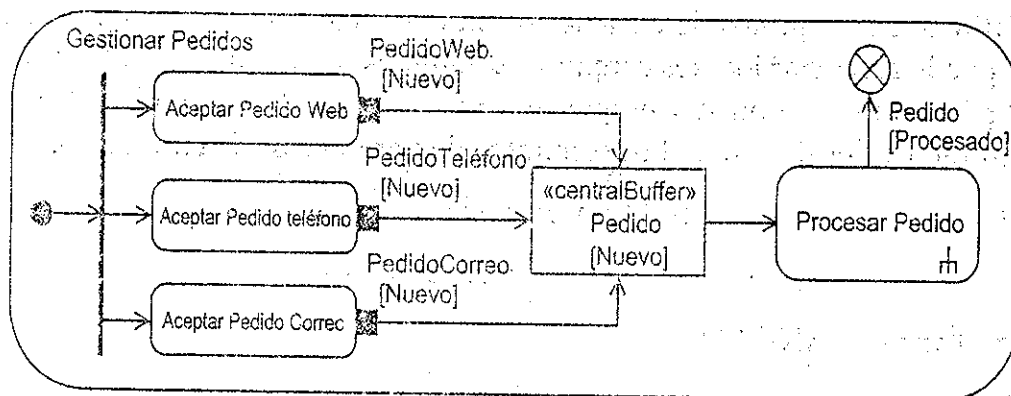


Figura 15.15.

15.12. Diagramas de visión de interacción

Los diagramas de visión de interacción, una forma especial de un diagrama de actividad, muestran interacciones y ocurrencias de interacción. Se utilizan para modelar el flujo de control de alto nivel entre interacciones. Tratamos las interacciones y los diagramas de interacción en detalle en el capítulo 12.

Un uso particularmente potente para los diagramas de visión de interacción es ilustrar el flujo de control entre casos de uso. Si representa cada caso de uso como una interacción, puede utilizar la sintaxis de diagramas de actividad para mostrar cómo el flujo de control se mueve entre ellos.

La figura 15.16 muestra un diagrama de visión de interacción, GestionarCursos, que muestra el flujo entre las interacciones de bajo nivel Conectarse, ObtenerOpciónCurso, EncontrarCurso, EliminarCurso y AñadirCurso. Cada una de estas interacciones representa un caso de uso por lo que el diagrama de visión de interacción captura el flujo de control entre estos casos de uso.

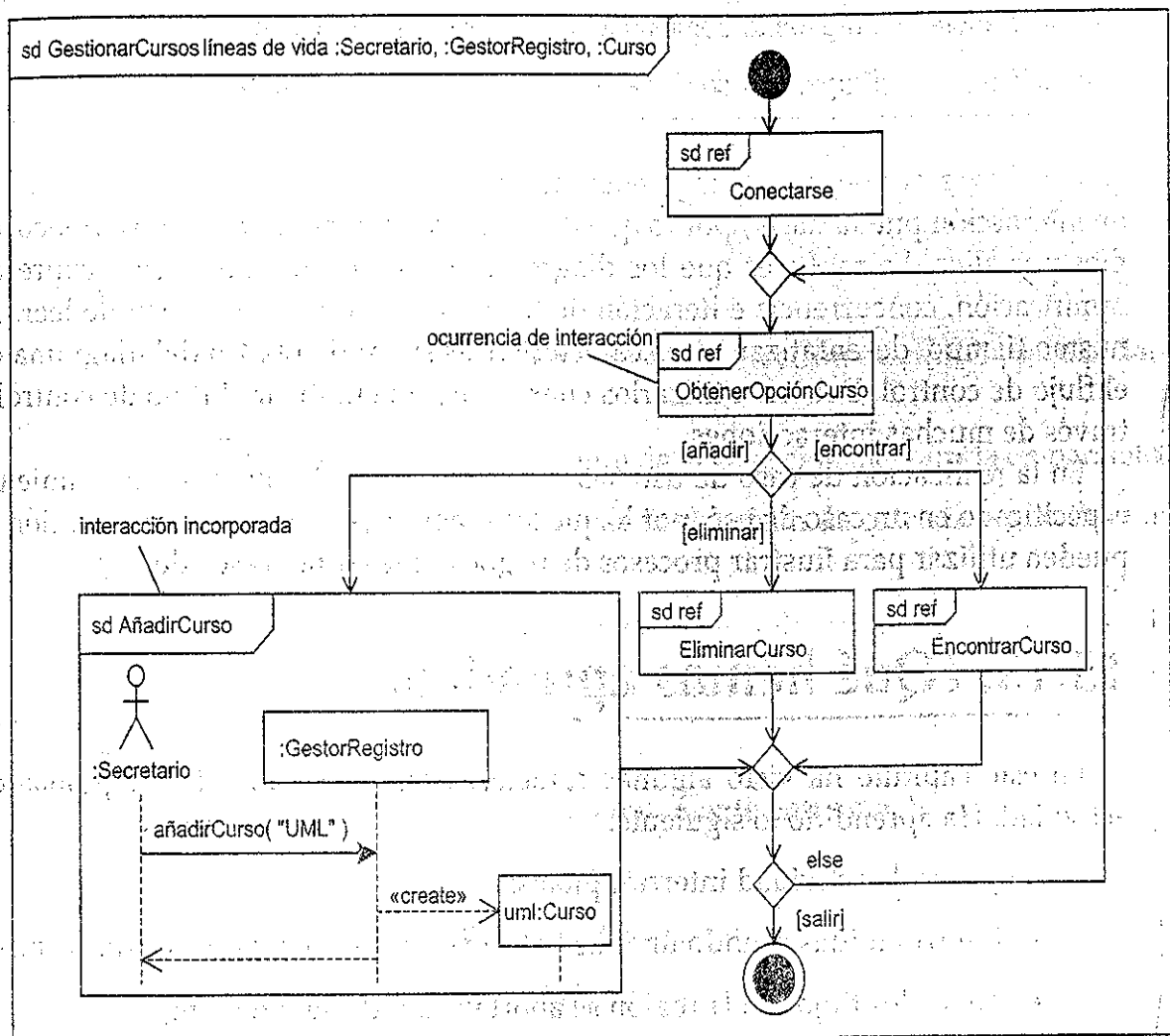


Figura 15.16.

Observe que las líneas de vida que participan en la interacción se pueden listar detrás de la palabra clave de su mismo nombre en el encabezado del diagrama. Esto puede ser documentación de utilidad ya que las líneas de vida se ocultan dentro de las ocurrencias de interacción.

Los diagramas de visión de interacción tienen la misma sintaxis que los diagramas de actividad excepto que muestran interacciones incorporadas y ocurrencias de interacción en lugar de nodos de actividad y objeto. Puede mostrar ramificación, concurrencia y bucle según se describe en la tabla 15.1. Esta tabla también propor-

ciona una visión de las diferencias entre la sintaxis de diagramas de secuencia y diagramas de visión de interacción.

Tabla 15.1.

Acción	Diagramas de secuencia	Diagrama de visión de interacción
Ramificación	Fragmentos combinados.	Nodo de decisión y nodo de fusión alt y opt.
Concurrencia	Fragmento combinado par.	Nodos fork y join.
Iteración	Fragmento combinado loop.	Ciclos en el diagrama.

Los diagramas de secuencia pueden hacer lo mismo que un diagrama de visión de interacción pueda hacer, por lo que debería preguntarse por qué nos preocupamos por ellos. La razón es que los diagramas de visión de interacción expresan ramificación, concurrencia e iteración de forma visual que es más fácil de leer. Al mismo tiempo, desenfatan otras características situando el foco del diagrama en el flujo de control. Debería utilizarlos cuando quiera enfatizar el flujo de control a través de muchas interacciones.

En la realización de caso de uso, las interacciones expresan el comportamiento especificado en un caso de uso, por lo que los diagramas de visión de interacción se pueden utilizar para ilustrar procesos de negocio que cortan casos de uso.

15.13. ¿Qué hemos aprendido?

En este capítulo ha visto algunas características avanzadas de diagramas de actividad. Ha aprendido lo siguiente:

- Regiones de actividad interrumpibles:
 - Interrumpidas cuando un token atraviesa un extremo que se interrumpe.
 - Todos los flujos en la región se abortan cuando se interrumpe.
 - Los extremos que se interrumpen se dibujan como una flecha de zigzag o como una flecha normal con un icono de zigzag sobre ésta.
- Pins de excepción:
 - Muestran como resultado un objeto de excepción a partir de una acción.
 - Se indican con un triángulo equilátero.
- Nodos protegidos:
 - Tienen un extremo que se interrumpe que lleva a un manejador de excepción.

- Se abortan cuando se alcanza una excepción de tipo correcto, y el flujo pasa al nodo del manejador de excepción.
- Son instantáneos.
- Nodos de expansión:
 - Representan una colección de objetos que fluyen en o fuera de una región de expansión.
 - La región se ejecuta una vez por elemento de entrada.
 - Restricciones:
 - El tipo de la colección de salida debe coincidir con el tipo de la colección de entrada.
 - El tipo del objeto contenido en las colecciones de entrada y salida deben ser lo mismo.
 - Modos:
 - Iterativo, procesa cada elemento de la colección de entrada secuencialmente.
 - Paralelo, procesa cada elemento de la colección de entrada en paralelo.
 - Flujo, procesa cada elemento de la colección de entrada según llega al nodo.
 - No existe modo predeterminado.
- Enviar señales y aceptar eventos:
 - Señales:
 - Información que se pasa asincrónicamente entre objetos.
 - Clase estereotipada <<signal>>.
 - La información se alberga en los atributos.
 - Nodo de acción de enviar señal:
 - Empieza cuando existe un token en todos los pin de entrada.
 - Se ejecuta, se construye y envía un objeto de señal.
 - Luego termina y ofrece tokens de control a sus extremos de salida.
 - Nodo de acción de aceptar evento:
 - Iniciado por un extremo entrante de control o si no existe extremo entrante, cuando se inicie la actividad que lo posee.
 - Espera un evento del tipo especificado:
 - Muestra como resultado un token que describe el evento.

- Continúa aceptando eventos mientras se ejecuta la actividad que lo posee.
- Para un evento de señal, el token de salida es una señal.
- Flujo avanzado de objeto:
 - Los efectos de entrada y salida muestran los efectos que tiene una acción en sus objetos de entrada y salida:
 - Escriba los efectos entre llaves junto al pin.
 - Selección, una condición en un flujo de objeto que hace que acepte solamente aquellos objetos que cumplen la condición:
 - Sitúe la condición de selección en una nota estereotipada <<selection>>, unida al flujo de objeto
 - Transformación: Transforma objetos en un flujo de objeto en un tipo diferente:
 - Sitúe la expresión de transformación en una nota estereotipada <<transformation>>, unida al flujo de objeto.
 - Multidifusión envía un objeto a muchos receptores:
 - Estereotipe el flujo de objeto <<multicast>>.
 - Multirreceptor recibe objetos de muchos emisores:
 - Estereotipe el flujo de objeto <<multireceive>>.
- Los conjuntos de parámetros permiten que una acción tenga conjuntos alternativos de pins de entrada y salida:
 - Los conjuntos de parámetros de entrada contienen pins de entrada.
 - Los conjuntos de parámetros de salida contienen pins de salida.
 - Solamente un conjunto de parámetros de entrada y un conjunto de parámetros de salida se pueden utilizar por cada ejecución de la acción.
- Nodo de buffer central: Los nodos de objeto que se utilizan específicamente como buffer:
 - Estereotipe el nodo de objeto <<centralBuffer>>.
- Los diagramas de visión de interacción muestran el flujo entre interacciones y ocurrencias de interacción:
 - Ramificación: Nodos de decisión y fusión.
 - Concurrencia: Nodos fork y join.
 - Iteración: Ciclos en el diagrama.